Best Practices - Speeding up deployments

You can choose rolling updates for your Amazon ECS service. Deployments might take longer than you expect, but you can modify a few options to speed up your deployments. For context, by choosing this deployment type, you tell the Amazon ECS service scheduler to replace any currently running tasks with new tasks whenever a new service deployment is started. The deployment configuration determines the specific number of tasks that Amazon ECS adds to or removes from the service during a rolling update. The following is an overview of the deployment process:

1. The scheduler starts your application.
2. The scheduler then decides if your application is ready for web traffic.
3. When you scale down or create a new version of the application, the scheduler decides whether your application is safe to stop. At the same time, it must maintain the availability of the application during the rolling deployment.

The strategy of maintaining task availability can cause deployments to take longer than you expect.

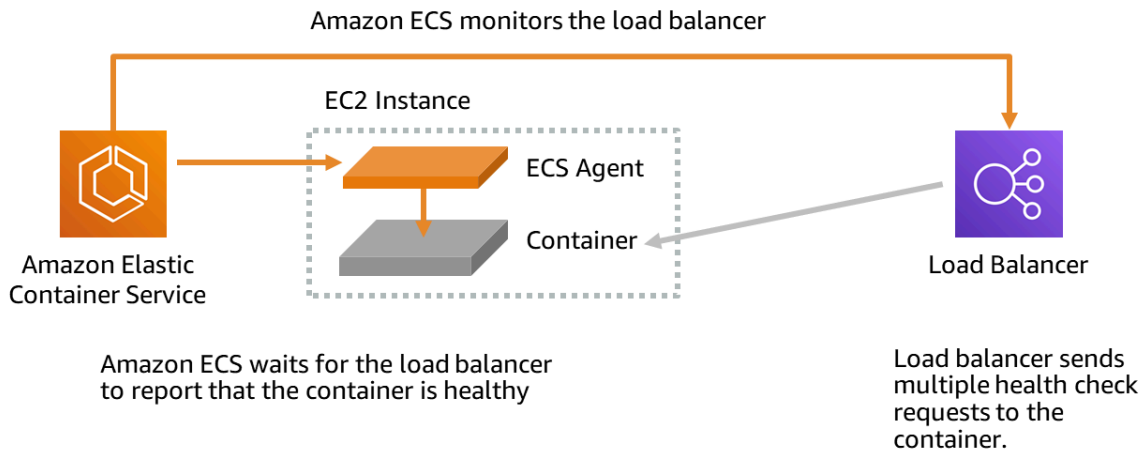
To speed up deployment times, modify the default load balancer, Amazon ECS agent, service and task definition options. The following sections detail how you can modify all of these to speed up deployments.

Topics

- Load balancer health check parameters
- Load balancer connection draining
- Container image type
- Container image pull behavior
- Task deployment

Load balancer health check parameters

The following diagram describes the load balancer health check process. The load balancer periodically sends health checks to the Amazon ECS container. The Amazon ECS agent monitors and waits for the load balancer to report on the container health. It does this before it considers the container to be in a healthy status.



Two Elastic Load Balancing health check parameters affect deployment speed:

- Health check interval: Determines the approximate amount of time, in seconds, between health checks of an individual container. By default, the load balancer checks every 30 seconds.

  This parameter is named:

  - `HealthCheckIntervalSeconds` in the Elastic Load Balancing API
  - **Interval** on the Amazon EC2 console
- Healthy threshold count: Determines the number of consecutive health check successes required before considering an unhealthy container healthy. By default, the load balancer requires five passing health checks before it reports that the target container is healthy.

  This parameter is named:

  - `HealthyThresholdCount` in the Elastic Load Balancing API
  - **Healthy threshold** on the Amazon EC2 console

With the default setting, the total time to determine the health of a container is two minutes and 30 seconds (`30 seconds * 5 = 150 seconds`).

You can speed up the health-check process if your service starts up and stabilizes in under 10 seconds. To speed up the process, reduce the number of checks and the interval between the checks.

- `HealthCheckIntervalSeconds` (Elastic Load Balancing API name) or **Interval** (Amazon EC2 console name): 5
- `HealthyThresholdCount` (Elastic Load Balancing API name) or **Healthy threshold** (Amazon EC2 console name): 2

With this setting, the health-check process takes 10 seconds compared to the default of two minutes and 30 seconds.

Setting the Elastic Load Balancing health check parameters to speed up deployment

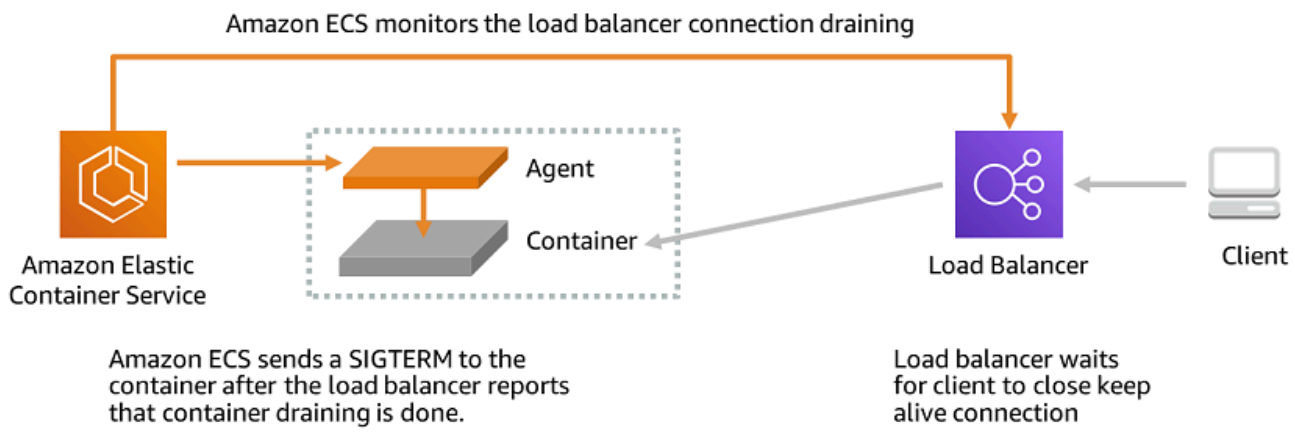
1. Go to the https://console.aws.amazon.com/ec2/.
2. From the left navigation, under **Load Balancing**, select **Target Groups**.
3. On the **Target groups** page, select the target group.
4. On the target group page, select the **Health checks** tab.
5. Click **Edit**.
6. Expand **Advanced health check settings**.
7. Set the health check parameters.
8. Click **Save changes**.

For more information about the Elastic Load Balancing health check parameters, see TargetGroup in the *Elastic Load Balancing API Reference*.

Load balancer connection draining

To allow for optimization, clients maintain a keep alive connection to the container service. This is so that subsequent requests from that client can reuse the existing connection. When you want to stop traffic to a container, you notify the load balancer.

The following diagram describes the load balancer connection draining process. When you tell the load balancer to stop traffic to the container, it periodically checks to see if the client closed the keep alive connection. The Amazon ECS agent monitors the load balancer and waits the load balancer to report that the keep alive connection is closed.



The amount of time that the load balancer waits is the deregistration delay. You can configure the following load balancer setting to speed up your deployments.

- `deregistration_delay.timeout_seconds`: 300 (default)

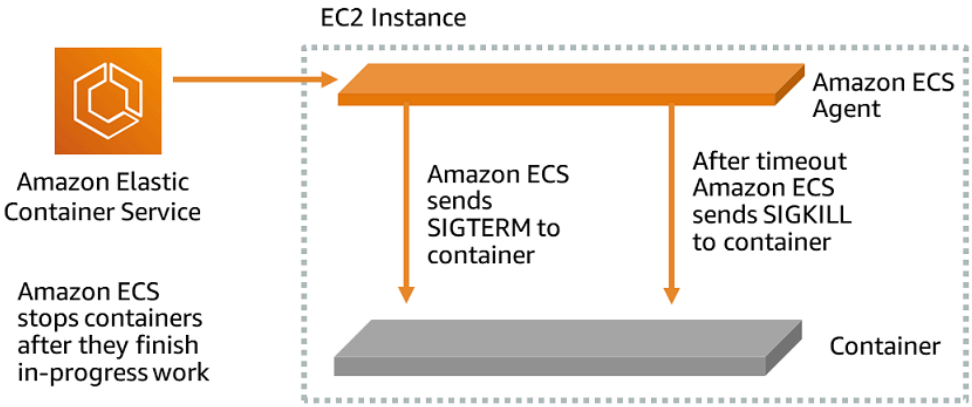
When you have a service with a response time that's under one second, set the parameter to the following value to have the load balancer only wait five seconds before it breaks the connection between the client and the backend service:

- `deregistration_delay.timeout_seconds`: 5

> **Note**
>
> Do not set the value to 5 seconds when you have a service with long-lived requests, such as slow file uploads or streaming connections.

SIGTERM responsiveness

The following diagram shows how Amazon ECS terminates a task. Amazon ECS first sends a SIGTERM signal to the task to notify the application needs to finish and shut down, and then Amazon ECS sends a SIGKILL message. When applications ignore the SIGTERM, the Amazon ECS service must wait to send the SIGKILL signal to terminate the process.



The amount of time that Amazon ECS waits is determined by the following Amazon ECS agent option:

- `ECS_CONTAINER_STOP_TIMEOUT`: 30 (default)

  For more information about the container agent parameter, see Container agent configuration in the *Amazon Elastic Container Service Developer Guide*.

To speed up the waiting period, set the Amazon ECS agent option to the following value:

> **Note**
>
> If your application takes more than one second, multiply the value by two and use that number as the value.

- `ECS_CONTAINER_STOP_TIMEOUT: 2`

In this case, the Amazon ECS waits two seconds for the container to shut down, and then Amazon ECS sends a SIGKILL message when the application didn't stop.

You can also modify the application code to trap the SIGTERM signal and react to it. The following is example in JavaScript:

```
process.on('SIGTERM', function() {
  server.close();
})
```

This code causes the HTTP server to stop listening for any new requests, finish answering any in-flight requests, and then the Node.js process terminates. This is because its event loop has nothing left to do. Given this, if it takes the process only 500 ms to finish its in-flight requests, it terminates early without having to wait out the stop timeout and get sent a SIGKILL.

Container image type

The time that it takes a container to start up varies, based on the underlying container image. For example, a fatter image (full versions of Debian, Ubuntu, and Amazon1/2) might take longer to start up because there are a more services that run in the containers compared to their respective slim versions (Debian-slim, Ubuntu-slim, and Amazon-slim) or smaller base images (Alpine).

Container image pull behavior

Container image pull behavior for Fargate launch types

Fargate does not cache images, and therefore the whole image is pulled from the registry when a task runs. The following are our recommendations for images used for Fargate tasks:

- Use a larger task size with additional vCPUs. The larger task size can help reduce the time that is required to extract the image when a task launches.
- Use a smaller base image.
- Have the repository that stores the image in the same Region as the task.

Container image pull behavior for Fargate Windows launch types

Fargate Windows caches the most recent month's, and the previous month's, servercore base image provided by Microsoft. These images match the KB/Build number patches updated each Patch Tuesday. For example, on 8/8/2023 Microsoft released KB5029247 (17763.4737) for Windows Server 2019. The previous month KB on 7/11/2023 was KB5028168 (17763.4645). So for the platforms `WINDOWS_SERVER_2019_CORE` and `WINDOWS_SERVER_2019_FULL` the following container images were cached:

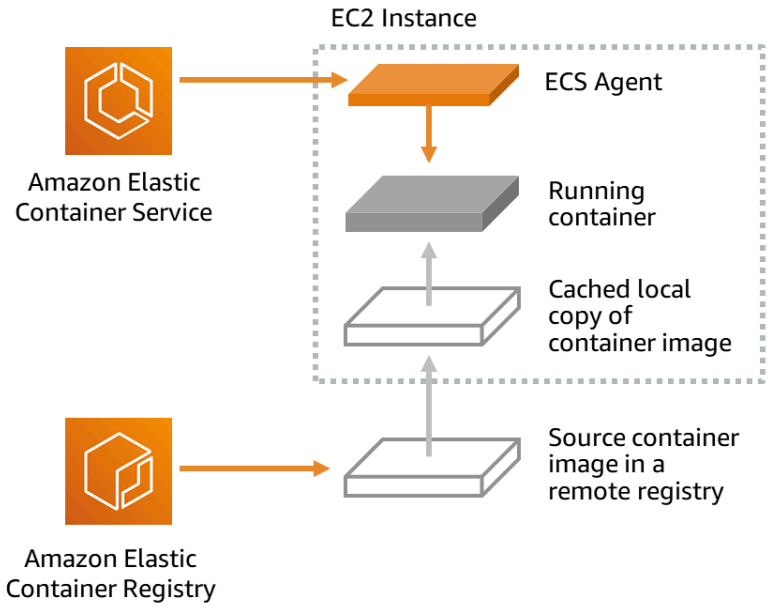
- `mcr.microsoft.com/windows/servercore:ltsc2019`
- `mcr.microsoft.com/windows/servercore:10.0.17763.4737`
- `mcr.microsoft.com/windows/servercore:10.0.17763.4645`

Additionally, on 8/8/2023 Microsoft released KB5029250 (20348.1906) for Windows Server 2022. The previous months KB on 7/11/2023 was KB5028171 (20348.1850). So for the platforms `WINDOWS_SERVER_2022_CORE` and `WINDOWS_SERVER_2022_FULL` the following container images were cached:

- `mcr.microsoft.com/windows/servercore:ltsc2022`
- `mcr.microsoft.com/windows/servercore:10.0.20348.1906`
- `mcr.microsoft.com/windows/servercore:10.0.20348.1850`

Container image pull behavior for Amazon EC2 launch types

When the Amazon ECS agent starts a task, it pulls the Docker image from its remote registry, and then caches a local copy. When you use a new image tag for each release of your application, this behavior is unnecessary.



The `ECS_IMAGE_PULL_BEHAVIOR` agent parameter determines the image pull behavior and has the following values:

- `ECS_IMAGE_PULL_BEHAVIOR: default`

  The image will be pulled remotely. If the pull fails, the cached image in the instance will be used.
- `ECS_IMAGE_PULL_BEHAVIOR: always`

  The image will be pulled remotely. If the pull fails, the task fails.

To speed up deployment, set the Amazon ECS agent parameter to one of the following values:

- `ECS_IMAGE_PULL_BEHAVIOR: once`

  The image is pulled remotely only if it wasn't pulled by a previous task on the same container instance or if the cached image was removed by the automated image cleanup process. Otherwise, the cached image on the instance is used. This ensures that no unnecessary image pulls are attempted.
- `ECS_IMAGE_PULL_BEHAVIOR: prefer-cached`

The image is pulled remotely if there is no cached image. Otherwise, the cached image on the instance is used. Automated image cleanup is disabled for the container to ensure that the cached image isn't removed.

Setting the parameter to either of the preceding values can save time because the Amazon ECS agent uses the existing downloaded image. For larger Docker images, the download time might take 10-20 seconds to pull over the network.

Task deployment

To ensure that there's no application downtime, the deployment process is as follows:

1. Start the new application containers while keeping the existing containers running.
2. Check that the new containers are healthy.
3. Stop the old containers.

Depending on your deployment configuration and the amount of free, unreserved space in your cluster it may take multiple rounds of this to complete replace all old tasks with new tasks.

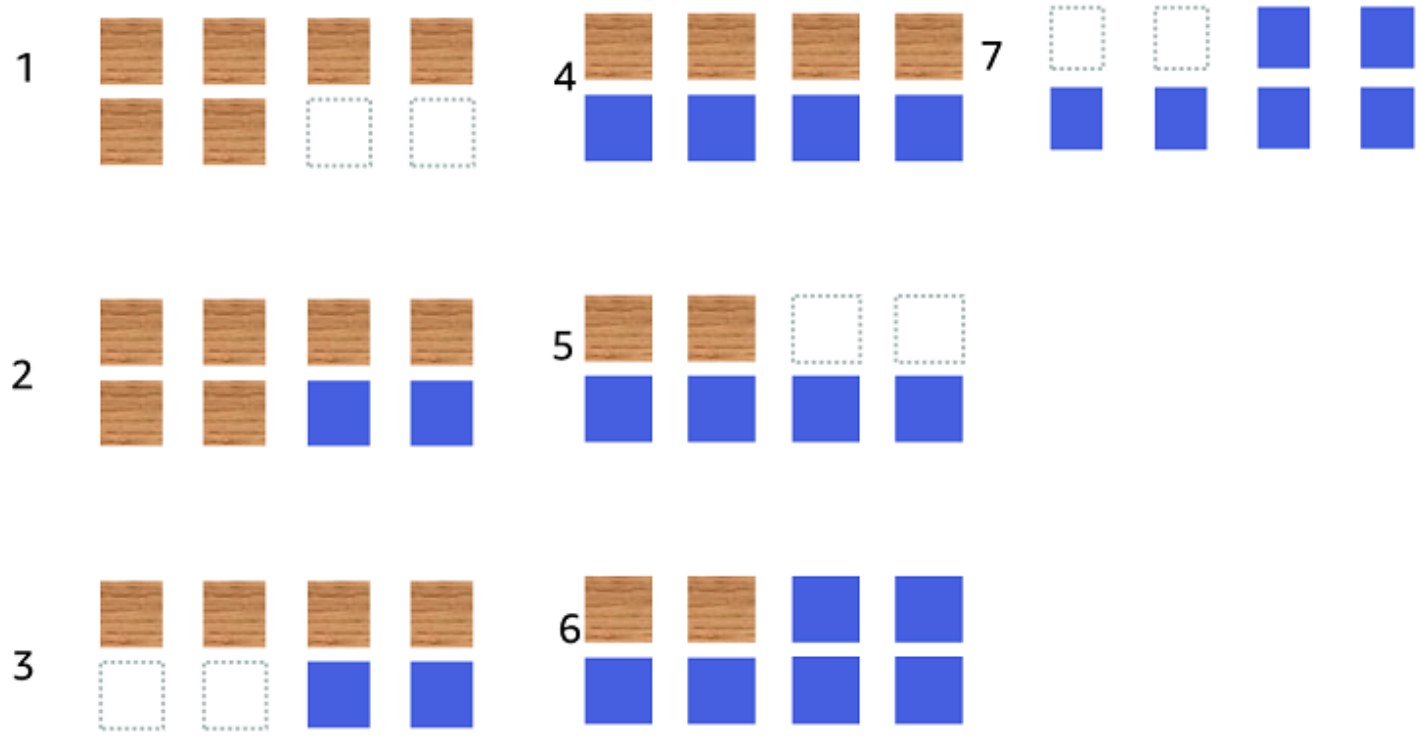
There are two ECS service configuration options that you can use to modify the number:

- `minimumHealthyPercent`: 100% (default)

  The lower limit on the number of tasks for your service that must remain in the `RUNNING` state during a deployment. This is represented as a percentage of the desiredCount. It's rounded up to the nearest integer. This parameter enables you to deploy without using additional cluster capacity.
- `maximumPercent`: 200% (default)

  The upper limit on the number of tasks for your service that are allowed in the `RUNNING` or `PENDING` state during a deployment. This is represented as a percentage of the desiredCount. It's rounded down to the nearest integer.

Consider the following service that has six tan tasks, deployed in a cluster that has room for eight tasks total. The default Amazon ECS service configuration options don't allow the deployment to go below 100% of the six desired tasks.



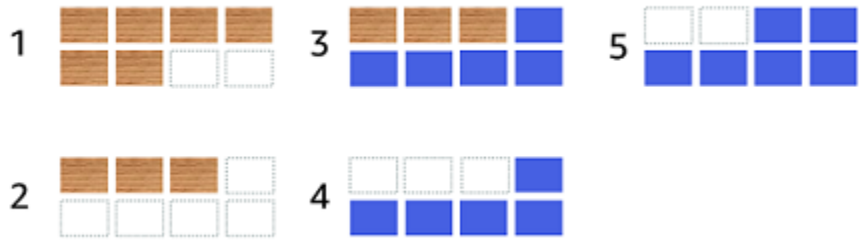
The deployment process is as follows:

1. The goal is to replace the tan tasks with the blue tasks.
2. The scheduler starts two new blue tasks because the default settings require that there are six running tasks.
3. The scheduler stops two of the tan tasks because there will be a total of six tasks (four tan and two blue).
4. The scheduler starts two additional blue tasks.
5. The scheduler shuts down two of the tan tasks.
6. The scheduler starts two additional blue tasks.
7. The scheduler shuts down the last two tan tasks.

In the above example, if you use the default values for the options, there is a 2.5 minute wait for each new task that starts. Additionally, the load balancer might have to wait 5 minutes for the old task to stop.

You can speed up the deployment by setting the `minimumHealthyPercent` value to 50%.

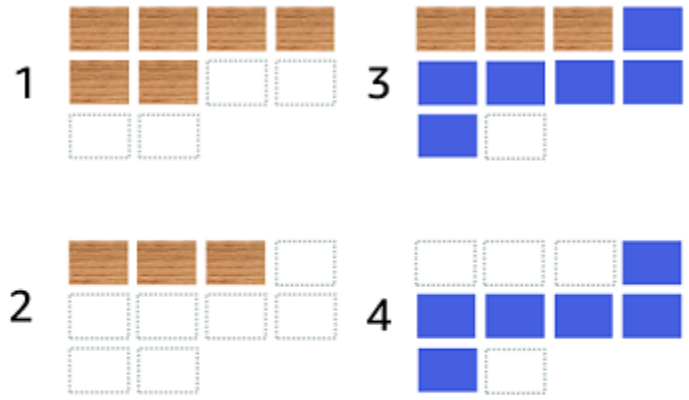
Consider the following service that has six tan tasks, deployed in a cluster that has room for eight tasks total.



The deployment process is as follows:

1. The goal is to replace the tan tasks with the blue tasks.
2. The scheduler stops three of the tan tasks. There are still three tan tasks running which meets the `minimumHealthyPercent` value.
3. The scheduler starts five blue tasks.
4. The scheduler stops the remaining three tan tasks.
5. The scheduler starts the final blue tasks.

You could also add additional free space so that you can run additional tasks.



The deployment process is as follows:

1. The goal is to replace the tan tasks with the blue tasks.
2. The scheduler stops three of the tan tasks
3. The scheduler starts six blue tasks
4. The scheduler stops the three tan tasks.

Use the following values for the ECS service configuration options when your tasks are idle for some time and don't have a high utilization rate.

- `minimumHealthyPercent`: 50%
- `maximumPercent`: 200%